AI + ML PROJECT SELECTION GUIDE

How to choose real-world, resume-worthy use cases and convert them into production-grade portfolio projects.

Personalized for Sai Ruthwik Reddy Kandimalla

Use case research | Dataset selection | Production engineering | Resume presentation

Best formula: Real company use case + legal public data + original implementation + production deployment + evaluation + documented trade-offs.

The Best Way to Select a Project

Yes, this is the strongest approach for selecting AI + ML projects for your resume. Do not begin with a random tutorial title. Begin with a genuine business problem that a real engineering team has solved, study the production constraints, obtain legal public data, and build a smaller but complete version.

Why this method works

- It gives the project a real user and measurable business purpose.
- It forces you to make architecture and evaluation decisions instead of copying code.
- It lets you discuss trade-offs, failures, security, scale, latency and cost in interviews.
- It demonstrates engineering ownership from data to deployment.
- It produces stronger resume evidence than a notebook-only model or a basic chatbot.

A project is resume-worthy when you can explain the problem, data, baseline, design choices, evaluation, failures, deployment and measurable results.

The three-resource method

1	Evidently ML and LLM System Design	Find real production use cases and company case studies.
2	Company engineering blogs	Study architecture, trade-offs, failures, scale and evaluation.
3	Kaggle, Hugging Face or public portals	Obtain legal datasets and establish a reproducible baseline.

Best Resources for Real Use Cases

1. Evidently AI: ML and LLM System Design

This should be your first stop. It organizes hundreds of real-world AI, ML and LLM case studies from many companies and supports browsing by use case and technology.

- Generative AI and LLMs
- RAG and AI agents
- Computer vision and NLP
- Recommendation systems
- Search and ranking
- Fraud detection and anomaly detection
- ML platforms, evaluation and production operations

Use it correctly

- Choose one domain.
- Read three case studies about the same problem.
- Record the user, business goal, data, architecture, metrics and main failure modes.
- Identify the smallest original version you can implement.

2. Curated ML system-design repositories

- A Curated List of ML System Design Case Studies: useful for browsing company, domain and production problem.
- ML Practical Use Cases: useful for searching RAG, agents, fraud, recommendations, ranking, forecasting, classification and computer vision.

3. Engineering blog directories

- Evidently list of machine-learning engineering blogs.
- Awesome ML Engineering Blogs directory.
- Prioritize technical posts describing one deployed system instead of product announcements.

Recommended company blogs

Production ML	Uber, Netflix, Airbnb, DoorDash, Pinterest, LinkedIn, Spotify and Amazon Science
LLM, RAG and agents	Microsoft, AWS, NVIDIA, Databricks, Dropbox, Uber, LinkedIn, OpenAI, Anthropic and Hugging Face
Indian-scale systems	Swiggy, Razorpay, PhonePe, Flipkart, Meesho and Zomato

Dataset and Problem Sources

Kaggle	Classical ML, forecasting, CV, competitions and business datasets	Do not submit only a notebook; add production engineering.
Hugging Face Datasets	NLP, transformers, LLM evaluation, speech and multimodal data	Read dataset cards and licenses carefully.
UCI Repository	Learning algorithms and building clean reproducible baselines	Often too small or clean for a flagship project.
Data.gov.in	Original India-specific projects in agriculture, transport, health and energy	Expect cleaning, inconsistent schemas and missing context.
Google Dataset Search	Finding niche datasets across academic and public sites	Verify the original source, ownership and license.
Papers with Code	Research-backed DL, NLP and CV projects	Do not only reproduce; modify and evaluate independently.

Kaggle is a starting point, not the finished project

Convert a competition or dataset into a production system:

Dataset -> validation -> training pipeline -> experiment tracking -> model registry -> FastAPI -> Docker -> cloud deployment -> monitoring -> retraining design

Using winning solutions correctly

- Build your own baseline first.
- Study strong solutions only after recording your baseline result.
- Extract reusable ideas such as validation strategy, feature engineering or ensembling.
- Reimplement the useful idea and measure whether it improves your system.
- Credit original ideas in your documentation.

Seven-Step Project Selection Workflow

Step 1 - Select one business domain

- Enterprise productivity
- Customer support
- E-commerce and retail
- Banking and fintech
- Healthcare operations
- Manufacturing
- Cybersecurity
- Supply chain
- Developer productivity

Best starting domains for your profile

- Enterprise productivity, because it matches agentic workflow automation.
- Customer support, because it combines classification, retrieval and agents.
- E-commerce, because it supports recommendation, forecasting and price modelling.
- Banking or fraud detection, because it demonstrates high-stakes ML, monitoring and explainability.

Step 2 - Read three real production case studies

Search for one problem category such as enterprise RAG, support automation, transaction fraud, recommendation ranking, demand forecasting, predictive maintenance, document intelligence or agentic workflow automation.

Step 3 - Write the business problem before coding

- Who is the user?
- What decision or task does the system support?
- What is wrong with the current process?
- What data is available?
- What output should the AI or ML system produce?
- What metric defines success?
- What is the consequence of a wrong output?

Seven-Step Workflow - Continued

Step 4 - Find legal public data

- Use Kaggle, UCI, Hugging Face, public APIs or government portals.
- Use synthetic data only when required and label it clearly.
- Verify license, privacy, personally identifiable information and commercial-use conditions.

Step 5 - Build a baseline

Simple preprocessing, a basic model, an appropriate business metric and error analysis.	Simple retrieval or single-agent workflow, a small evaluation set, and quality, latency, safety and cost measurements.

Step 6 - Add production engineering

- Modular Python and configuration management
- FastAPI service and structured validation
- PostgreSQL and a vector database when justified
- Redis when caching, rate limiting or state requires it
- Unit, integration and end-to-end tests
- Docker and CI/CD
- Cloud deployment
- Evaluation and observability
- Security controls and failure handling

Step 7 - Present measurable evidence

Weak: Developed an AI-powered chatbot using LangChain.

Strong: Built and deployed a citation-grounded support assistant using LangGraph, hybrid retrieval and reranking; evaluated it on a versioned question set and added regression tests, prompt-injection controls, tracing, caching and provider fallback.

Only include numbers that you measured honestly. Report the dataset size, evaluation-set size, model metric, latency, cost, error reduction or reliability improvement with the test method.

Best Three-Project Portfolio Mix

Project 1 - Production classical ML system

Recommended ideas: transaction fraud detection, customer churn, demand forecasting, predictive maintenance or support-ticket classification.

- Data validation and feature pipeline
- Baseline and multiple model comparison
- MLflow tracking and model registry
- FastAPI for online and batch predictions
- Docker, CI/CD and AWS deployment
- Drift monitoring and retraining design

Project 2 - Production RAG or agentic system

Recommended ideas: enterprise knowledge and allocation agent, support-resolution agent, technical research assistant or compliance-document reviewer.

- LangGraph workflow, controlled tools and human approval
- Hybrid retrieval, reranking and verifiable citations
- MCP integration where it provides real value
- Evaluation, guardrails and observability
- Caching, provider fallback and failure recovery

Project 3 - Deep-learning or multimodal system

Recommended ideas: manufacturing defect detection, document understanding, product matching or multimodal support-ticket triage.

- PyTorch and transfer learning
- Data augmentation and error analysis
- Model-serving API
- Latency and resource measurements
- Docker and cloud deployment

Project Evaluation Scorecard

Score every idea from 0 to 2 on each criterion. Prefer ideas scoring at least 16 out of 20.

Business value	No clear user	Plausible value	Clear user and decision
Originality	Common tutorial	Modified tutorial	Original framing/data
Data realism	Tiny clean data	Some complexity	Messy, temporal or multi-table
Evaluation	Only demo output	One metric	Business + technical metrics
Engineering depth	Notebook only	API or Docker	Tested, deployed pipeline
Reliability	Happy path only	Basic errors	Retries, fallback, recovery
Security/privacy	Ignored	Mentioned	Implemented controls
Monitoring	None	Logs only	Quality, latency, cost, drift
Explainability	Cannot defend choices	Partial reasoning	Clear trade-offs and limits
Demo quality	Code only	README	README, demo and architecture

Reject or downgrade a project if

- It is copied from a tutorial with only cosmetic changes.
- The dataset license is missing or unclear.
- There is no meaningful evaluation set or metric.
- The architecture is unnecessarily complex only to list tools.
- You cannot explain what happens when the model is wrong.
- You cannot reproduce the training or deployment process.

Resume and Interview Packaging

Publish these artifacts

- Problem statement and intended users
- Architecture diagram
- Data card and license information
- Reproducible setup instructions
- Training and evaluation pipeline
- API documentation
- Automated test results
- Latency, cost and quality measurements
- Threat model and failure analysis
- Deployment and rollback instructions
- Short demo video

Write resume bullets using this pattern

Action + system built + technical decisions + evaluated result + production capability

Example structure

Built and deployed a production [system] using [core architecture]; improved/measured [honest metric] on [evaluation set], while adding [testing, monitoring, security or reliability controls].

Be ready to answer

- Why was AI or ML required?
- Why did you choose this model or workflow?
- What baseline did you compare against?
- How did you prevent leakage or hallucinations?
- How did you evaluate quality?
- What failed during development?
- How does the system handle outages, bad inputs and high latency?
- How would you scale it?
- What would you improve with real company data?

Final Checklist

- I started from a real business use case, not a tutorial title.
- I studied at least three related company case studies.
- I documented the user, decision, risk and success metric.
- I verified the dataset license and privacy conditions.
- I built and measured a simple baseline.
- I made original architecture or modelling decisions.
- I added appropriate backend, data and production components.
- I tested normal and failure cases.
- I evaluated quality, latency, cost and safety where relevant.
- I deployed the system and documented operations.
- I can explain limitations and trade-offs.
- My resume claims are measurable and reproducible.

Final recommendation: Use Evidently to discover the problem, company engineering blogs to understand production design, and Kaggle/Hugging Face/public portals to obtain data. Then build your own smaller end-to-end system.

Key resource links

- Evidently ML and LLM System Design: https://www.evidentlyai.com/ml-system-design
- Curated ML System Design Case Studies: https://github.com/Engineer1999/A-Curated-List-of-ML-System-Design-Case-Studies
- ML Practical Use Cases: https://github.com/mallahyari/ml-practical-usecases
- Awesome ML Engineering Blogs: https://primaprashant.github.io/ml-engineering-blogs/
- Kaggle Datasets: https://www.kaggle.com/datasets
- Kaggle Competitions: https://www.kaggle.com/competitions
- MLOps Community: https://mlops.community/blog

Prepared from the project-selection strategy and resource list provided in this conversation. Resource availability and dataset licenses should be rechecked before each project.